

Value Category

值类别

值类别是 C++ 中一个非常重要的概念，虽然在算法竞赛中可能用处不大，但了解它可以帮助我们发现并避免不必要的复制，从而提高代码的效率和性能。

值类别的概念在 C 语言、C++98、C++11 和 C++17 中经历了多次发展，逐渐成为一个较为复杂的概念。

不必要的复制

我们考虑将字符串塞入 vector 这一过程：

```
1 int main() {
2     std::vector<std::string> vec;
3     vec.reserve(3);
4     for (int i = 0; i < 3; ++i) {
5         std::string str;
6         std::cin >> str;
7         vec.push_back(str);
8     }
9     return 0;
10 }
```

可以发现字符串在转移的过程中，在 `str` 和 `vec` 中各保存了一份，内存占用加倍。

如果非要省下这一部分的内存，我们可以实现一个简陋的移动操作：自定义 `MyString` 结构体，内有一指针指向我们的字符串，即我们只需要把指针复制过去，并小心地清理原对象的指针，防止被错误析构。

```
1 struct MyString {
2     char *beg, *end;
3     // ...
4 };
5
6 void move_to(MyString &src, MyString &dst) {
7     dst.beg = src.beg;
8     dst.end = src.end;
9     src.beg = src.end = nullptr;
10 }
```

由于这种高效转移对象的需求较为常见，且与 C++ 的构造、析构等操作交互困难，C++11 将移动语义引入了语言核心。

C 语言中的值类别

在 C 语言标准中，对象是一个比变量更为一般化的概念，它指代一块内存区域，具有内存地址。对象的主要属性包括：大小、有效类型、值和标识符。标识符即变量名，值是该内存以其类型解释时的含义。例如，`int` 和 `float` 类型虽然都占用 4 字节，但对于同一块内存，我们会解释出不同的含义。

C 语言中每个表达式都具有类型和值类别。值类别主要分为三类：

- 左值 (lvalue)：隐含指代一个对象的表达式。即我们可以对该表达式取地址。
- 右值 (rvalue)：不指代对象的表达式，即指代没有存储位置的值，我们无法取该值的地址。

- 函数指代符：函数类型的表达式。

因此，只有可修改的左值（没有 `const` 修饰且非数组的左值）可以位于赋值表达式左侧。

对于某个要求右值作为它的操作数的运算符，每当左值被用作操作数，都会对该表达式应用左值到右值，数组到指针，或者函数到指针标准转换以将它转换成右值。

常见误区：

- 右值表达式继续运算可能是左值。例如 `int *a`，表达式 `a + 1` 是右值，但 `*(a + 1)` 是左值。
- 表达式才有值类别，变量没有。例如 `int *a`，不能说变量 `a` 是左值，可以说其在表达式 `a` 中做左值，

C++98 中的值类别

C++98 在值类别方面与 C 语言几乎一致，但增加了一些新的规则：

- 函数为左值，因为可以取地址。
- 左值引用（`T&`）是左值，因为可以取地址。
- 仅有 `const T&` 可绑定到右值。

复制消除

C++ 允许编译器执行复制消除（Copy Elision），可以减少临时对象的创建和销毁。

例如下面的代码，就触发了复制消除中的返回值优化（Return Value Optimization, RVO），你只会看到一次构造和一次复制构造，即便构造与析构有副作用。

```
1 struct X {
2     X() { std::puts("X::X()"); }
3
4     X(const X &) { std::puts("X::X(const X &)"); }
5
6     ~X() { std::puts("X::~~X()"); }
7 };
8
9 X get() {
10     X x;
11     return x;
12 }
13
14 int main() {
15     X x = get();
16     X y = X(X(X(X(x))));
17     return 0;
18 }
```

C++11 中的值类别

C++11 引入了移动语义和右值引用（`T&&`），包括移动构造、移动赋值函数。这给了我们利用临时对象的方法。

我们上面的 `move_to` 可以改写如下：

```

1 struct MyString {
2     // ...
3     MyString(MyString&& other) {
4         beg = other.beg;
5         end = other.end;
6         other.beg = other.end = nullptr;
7     }
8 };

```

我们现在关注的表达式特性增加了一点：

- 是否具有身份：是否指代一个对象，即是否有地址。
- 是否可被移动：是否具有移动构造、移动赋值等函数，让我们有办法利用这些临时对象。

因此我们有三种值类别：

- 有身份，不可移动：左值（lvalue）。
- 有身份，可被移动：亡值（xvalue）。
- 无身份，可被移动：纯右值（prvalue）。
- 无身份，不可移动：此类表达式无法使用。

另外 C++11 还引入了两个复合类别：

- 具有身份：泛左值（glvalue），即左值和亡值。
- 可被移动：右值（rvalue），即纯右值和亡值。

std::move

为了配合移动语义，C++11 还引入了一个工具函数 `std::move`，其作用是将左值强制转换为右值，以便触发移动语义。

```

1 int main() {
2     std::vector<int> a = {1, 2, 3};
3     std::cout << "a: " << a.data() << std::endl;
4     std::vector<int> b = a;
5     std::cout << "b: " << b.data() << std::endl;
6     std::vector<int> c = std::move(b);
7     std::cout << "c: " << c.data() << std::endl;
8 }

```

因此我们只需将 `push_back(str)` 改为 `push_back(std::move(str))` 即可避免复制。

```

1 int main() {
2     std::vector<std::string> vec;
3     vec.reserve(3);
4     for (int i = 0; i < 3; ++i) {
5         std::string str;
6         std::cin >> str;
7         vec.push_back(std::move(str));
8         // 另一种巧妙的写法，需要 C++17
9         // std::cin >> vec.emplace_back();
10    }
11    return 0;
12 }

```

由于 `std::string` 有小对象优化（Small String Optimization, SSO），短字符串直接存储于结构体内，你可能得输入较长的字符串才能观察到 `data` 指针的不变性。

C++17 中的值类别

C++17 进一步简化了值类别：

- 左值 (lvalue)：有身份，不可移动。
- 亡值 (xvalue)：有身份，可以移动。
- 纯右值 (prvalue)：对象的初始化。

C++11 将复制消除扩展到了移动上，下面的代码中 `urvo` 在编译器启用 RVO 的情况下是没有移动的。

C++17 要求纯右值非必须不实质化，直接构造到其最终目标的存储中，在构造之前对象尚不存在。因此在 C++17 中我们就没有返回这一步，也就不必依赖 RVO。也可以理解为强制了 URVO (Unnamed RVO)，但对于 NRVO (Named RVO) 还是非强制的。

```
1 std::string urvo() { return std::string("123"); }
2
3 std::string nrvo() {
4     std::string s;
5     s = "123";
6     std::cout << s;
7     return s;
8 }
9
10 int main() {
11     std::string str = urvo(); // 直接构造
12     std::string str = nrvo(); // 不一定直接构造，依赖于优化
13 }
```

同时 C++17 引入了临时量实质化的机制，当我们需要访问成员变量、调用成员函数等需要泛左值的情形时，可以隐式转换为亡值。

常见误区

下面的例子中：

- 在 `f1` 中返回 `std::move(x)` 是多余的，并不会带来性能上的提升，反而会干扰编译器进行 NRVO 优化。
- 在 `f2` 中返回 `std::move(x)` 是危险的，函数返回右值引用指向了已被销毁的局部变量 `s`，出现了悬空引用问题。

```
1 std::string f1() {
2     std::string s = "123";
3     // 等价于 return std::string(std::move(s))
4     return std::move(s);
5 }
6
7 std::string&& f2() {
8     std::string s = "123";
9     return std::move(s);
10 }
```

参考文献与推荐阅读

1. [Value categories](#)
 2. [Wording for guaranteed copy elision through simplified value categories](#)
 3. [C++ 中的值类别](#)
 4. [C++ 的右值引用、移动和值类别系统，你所需要的一切](#)
 5. [Copy elision](#)
-

本页面最近更新：2024/10/5 19:24:57, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [CCXXI](#), [CoderOJ](#), [lr1d](#), [ksyx](#), [mingyEx](#), [Ranganhar](#), [rogeryoungh](#), [Tiphereth-A](#), [Xeonacid](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用